RAG-Based Customer Support Assistant

Low-Level Design (LLD)

TABLE OF CONTENTS

1. Introduction
2. Module-Level Design
3. Data Structures
4. Workflow Design (LangGraph)
5. Conditional Routing Logic
6. HITL Design
7. API/Interface Design
8. Error Handling

1. INTRODUCTION

1.1 Purpose

This document provides the low-level implementation details for the RAG-based Customer Support Assistant. It describes internal module designs, data structures, algorithms, and implementation logic.

1.2 Scope

This LLD covers:

- Detailed module specifications
- Data structure definitions
- LangGraph workflow implementation
- Routing logic and decision criteria
- HITL escalation mechanisms
- Error handling strategies

1.3 Document Conventions

- Code snippets are shown in monospace font
- Module names are capitalized
- Function names use snake_case
- Class names use PascalCase

2. MODULE-LEVEL DESIGN

2.1 Document Processing Module

Module Name: ingest.py

Purpose: Handle PDF ingestion and preprocessing

Function: load_pdf(pdf_path: str)

Input: Path to PDF file (string)

Output: List of Document objects

Process:

1. Initialize PyPDFLoader with file path
2. Call loader.load() to extract pages
3. Return list of page objects with content and metadata

Error Handling: FileNotFoundError if PDF doesn't exist

Function: split_into_chunks(pages: List[Document])

Input: List of document pages

Output: List of text chunks

Process:

1. Initialize RecursiveCharacterTextSplitter
- chunk_size = 500 characters
- chunk_overlap = 50 characters
- separators = ["\n\n", "\n", ".", " "]
2. Call splitter.split_documents(pages)
3. Return list of smaller chunks

Algorithm: Recursive splitting tries separators in order until chunk size is met

2.2 Chunking Module

Module: RecursiveCharacterTextSplitter (LangChain)

Parameters:

- chunk_size: 500

- chunk_overlap: 50
- separators: ["\n\n", "\n", ".", " "]

Chunking Logic:

1. Try splitting on "\n\n" (paragraph breaks)
2. If chunks still too large, split on "\n" (line breaks)
3. If still too large, split on "." (sentences)
4. If still too large, split on " " (words)
5. Maintain 50-character overlap between adjacent chunks

Example:

Input: 1200-character text

Output: 3 chunks of ~500 chars each with 50-char overlap

Chunk 1: chars 0-500
Chunk 2: chars 450-950 (overlaps with chunk 1)
Chunk 3: chars 900-1200 (overlaps with chunk 2)

2.3 Embedding Module

Module: HuggingFaceEmbeddings

Model: all-MiniLM-L6-v2

Function: embed_documents(texts: List[str])

Input: List of text strings

Output: List of 384-dimensional vectors (float arrays)

Process:

1. Load pre-trained model from HuggingFace
2. Tokenize input texts
3. Pass through transformer model
4. Extract embeddings from final layer
5. Normalize vectors to unit length
6. Return as numpy arrays

Function: embed_query(query: str)

Input: Single query string

Output: Single 384-dimensional vector

Process: Same as embed_documents but for one input

2.4 Vector Storage Module

Module: ChromaDB (Chroma.from_documents)

Storage Location: ./chroma_db/

Function: from_documents(documents, embedding, persist_directory)

Input:

- documents: List of text chunks
- embedding: Embedding function
- persist_directory: Path to storage folder

Output: Chroma vectorstore object

Process:

1. Create ChromaDB collection

2. For each document:

   a. Generate embedding using embedding function
   b. Store (id, embedding, text, metadata) in collection

3. Persist to disk at persist_directory

4. Return vectorstore handle

Storage Format:

- SQLite database for metadata
- HDF5 files for vectors
- Automatically indexed for similarity search

2.5 Retrieval Module

Function: similarity_search(query: str, k: int)

Input:

- query: User's question (string)
- k: Number of results to return (integer)

Output: List of Document objects (top-k matches)

Process:

1. Convert query to embedding vector (384-dim)
2. Compute cosine similarity with all stored vectors
3. Sort by similarity score (descending)
4. Return top-k documents with highest scores

Similarity Metric: Cosine Similarity

Formula: similarity = (A · B) / (||A|| * ||B||)

Range: -1 to 1 (higher = more similar)

2.6 Query Processing Module

Module: graph.py

Contains all LangGraph node functions

Node Function: retriever_node(state: State)

Input: State object with user question

Output: Updated state with retrieved chunks

Process:

1. Extract question from state["question"]
2. Load ChromaDB vectorstore
3. Call vectorstore.similarity_search(question, k=3)
4. Extract page_content from each result
5. Store chunks in state["chunks"]
6. Return updated state

Node Function: generator_node(state: State)

Input: State object with chunks

Output: Updated state with generated answer

Process:

1. Extract chunks from state["chunks"]
2. If chunks exist:
    a. Concatenate chunks with "\n\n" separator
    b. Add prefix "Based on our knowledge base:\n\n"
3. If no chunks:
    a. Set answer to "I don't have enough information..."
4. Store answer in state["answer"]
5. Return updated state

Node Function: confidence_checker_node(state: State)

Input: State object with answer

Output: Updated state with confidence and needs_human flag

Process:

1. Extract answer from state["answer"]
2. Apply confidence rules (see Section 5)
3. Set state["confidence"] = "high" or "low"
4. Set state["needs_human"] = True or False
5. Return updated state

Node Function: hitl_node(state: State)

Input: State object for escalated query

Output: State object (unchanged in current implementation)

Process:

1. Log question to console
2. Log attempted answer to console
3. Print HITL escalation message
4. (In production: Send to ticketing system)
5. Return state

2.7 Graph Execution

Module Function: create_graph()

Output: Compiled LangGraph application

Process:

1. Initialize StateGraph with State schema
2. Add nodes:
   workflow.add_node("retriever", retriever_node)
   workflow.add_node("generator", generator_node)
   workflow.add_node("confidence", confidence_checker_node)
   workflow.add_node("hitl", hitl_node)
3. Define edges (sequential flow):
   workflow.set_entry_point("retriever")
   workflow.add_edge("retriever", "generator")
   workflow.add_edge("generator", "confidence")
4. Add conditional edge after confidence:
   workflow.add_conditional_edges(
   "confidence",
   should_escalate,
   {"hitl": "hitl", "end": END}
)
5. Add edge from HITL to END:
   workflow.add_edge("hitl", END)
6. Compile graph
7. Return compiled application

2.8 HITL Module

Current Implementation: Console logging

Production Requirements:

- Database to store escalated queries
- API integration with ticketing systems
- Email/Slack notifications
- Agent assignment logic
- Response tracking

3. DATA STRUCTURES

3.1 Document Representation

Class: Document (from LangChain)

Fields:

- page_content: str - The text content
- metadata: dict - Page number, source file, etc.

3.2 Chunk Format

Structure: List of Document objects

Each chunk:

- Text: 500 characters (approximate)
- Overlap: 50 characters with adjacent chunks
- Metadata: Source page, chunk index

3.3 Embedding Structure

Type: numpy.ndarray

Shape: (384,)

Data Type: float32

Range: Normalized to unit length

3.4 State Object for Graph

Class: State (TypedDict)

Fields:

- question: str - User's input question
- chunks: List[str] - Retrieved text chunks
- answer: str - Generated response
- confidence: str - "high" or "low"
- needs_human: bool - HITL escalation flag

4. WORKFLOW DESIGN (LANGGRAPH)

4.1 Node Definitions

Node 1: Retriever

Type: Processing Node

Input: State with question

Output: State with chunks populated

Responsibility: Search ChromaDB and retrieve relevant chunks

Dependencies: ChromaDB, Embedding Model

Node 2: Generator

Type: Processing Node

Input: State with chunks

Output: State with answer populated

Responsibility: Format chunks into coherent answer

Dependencies: None (expandable to LLM)

Node 3: Confidence Checker

Type: Decision Node

Input: State with answer

Output: State with confidence and needs_human flags

Responsibility: Evaluate answer quality

Dependencies: None (rule-based logic)

Node 4: HITL

Type: Processing Node

Input: State (escalated query)

Output: State (unchanged)

Responsibility: Log escalation, notify human agent Dependencies: Logging system (expandable to ticketing API)

4.2 Edge Definitions

Sequential Edges:

- START → Retriever
- Retriever → Generator
- Generator → Confidence Checker

Conditional Edge:

Confidence Checker → Decision:

- If needs_human = True → HITL Node
- If needs_human = False → END

Terminal Edges:

- HITL → END

4.3 State Transitions

Transition 1: START → Retriever

Transition 2: Retriever → Generator

Transition 3: Generator → Confidence

Transition 4a: Confidence → END (High Confidence)

Transition 4b: Confidence → HITL (Low Confidence)

5. CONDITIONAL ROUTING LOGIC

5.1 Answer Generation Criteria

Success Criteria:

- At least 1 relevant chunk retrieved
- Answer length > 50 characters
- Answer does not contain "I don't have enough information"

Failure Triggers:

- No chunks retrieved from ChromaDB
- All retrieved chunks have low similarity scores
- Answer is too generic or vague

5.2 Escalation Criteria

Rule 1: Missing Information

Trigger: Answer contains phrase "I don't have enough information"

Action: Set needs_human = True, confidence = "low"

Rule 2: Short Answer

Trigger: Answer length < 50 characters

Action: Set needs_human = True, confidence = "low"

Rationale: Very short answers likely lack sufficient detail

Rule 3: Low Retrieval Quality

Trigger: No chunks retrieved OR all similarity scores < 0.3

Action: Set needs_human = True, confidence = "low"

Rationale: Poor retrieval indicates question is outside knowledge base

Rule 4: High Confidence (Default)

Trigger: None of the above conditions met

Action: Set needs_human = False, confidence = "high"

5.3 Routing Decision Function

Function: should_escalate(state: State) -> str

Input: State object with confidence and needs_human fields

Output: String ("hitl" or "end")

Logic:

```
if state["needs_human"] == True:
    return "hitl"
else:
    return "end"
```

6. HITL DESIGN

6.1 When Escalation is Triggered

Trigger Conditions:

1. Confidence checker determines low confidence
2. Answer contains uncertainty phrases
3. Answer is too short (< 50 chars)
4. No relevant chunks found in retrieval

Non-Trigger Conditions:

- User asks follow-up questions (handled automatically)
- Answer is long but generic (needs better detection in future)

6.2 What Happens After Escalation

Current Implementation:

1. HITL node logs the following to console:
    - Original question - Bot's attempted answer
    - Escalation timestamp
2. User is notified that question was escalated
3. Conversation can continue with new questions

Production Implementation:

1. Create ticket in support system (Zendesk/Freshdesk):
    - Ticket ID generated
    - Question attached
    - Bot's attempt attached for context
2. Notify human agent:
    - Slack message to #support-escalations
    - Email to support@company.com
    - In-app notification to available agents
3. Agent assignment:
    - Round-robin distribution
    - Priority-based routing (VIP customers first)
    - Skill-based matching (technical vs. billing)
4. Response tracking:
    - Agent response time logged
    - Resolution time measured
    - Quality score collected (customer feedback)

6.3 How Human Response is Integrated

Current Implementation:

- Human response is manual (outside the system)
- No automatic integration back to user

Production Implementation:

1. Agent responds via ticketing system
2. System captures agent's response
3. Response stored in conversation history
4. User notified via email/SMS/chat
5. Conversation log updated with human answer
6. (Optional) Human answer added to knowledge base for future queries

Integration Flow:

User Query → Bot Attempt (Low Conf) → HITL Escalation → Agent Response → User Notification → Conversation Closed

6.4 HITL Data Schema

Escalation Record:


```
{
    "escalation_id": "ESC-2026-001234",
    "timestamp": "2026-04-24T14:32:00Z",
    "user_query": "How do I cancel my subscription?",
    "bot_answer":
    "I don't have enough information...",
    "confidence": "low",
    "retrieved_chunks": [...], "assigned_agent": "
    agent_42", "status": "pending",
    "resolution_time": null,
    "agent_response": null
}
```

7. API/INTERFACE DESIGN

7.1 Input Format

Function: ask_question(question: str)

Input: String (user's question)

Example: "How do I reset my password?"

Validation:

- Must not be empty string
- Maximum length: 500 characters
- Must contain at least one alphanumeric character

7.2 Output Format

Return Type: Dictionary (State object)

Fields:

- question: Original user query
- answer: Generated response
- confidence: "high" or "low"
- needs_human: True or False

7.3 Interaction Flow

CLI Interaction:

1. System displays welcome message
2. User types question
3. System displays "Processing..." indicator
4. System calls ask_question(user_input)
5. System displays answer
6. If HITL triggered, display escalation message
7. Loop back to step 2

8. ERROR HANDLING

8.1 Missing Data Errors

Error: PDF file not found

Trigger: FileNotFoundError in load_pdf()

Handling:

- Catch exception
- Print error message: "Error: PDF file not found at path X"
- Exit gracefully with instructions to check file path

Error: ChromaDB not initialized

Trigger: Attempting retrieval before ingestion

Handling:

- Check if ./chroma_db/ directory exists
- If not, print: "Error: Knowledge base not found. Please run ingest.py first."
- Exit with status code 1

8.2 No Relevant Chunks Found

Error: similarity_search() returns empty list

Trigger: Query has no semantic match in knowledge base

Handling:

- Generator node detects empty chunks list
- Sets answer = "I don't have enough information to answer that."
- Confidence checker flags for HITL escalation
- User receives appropriate message

8.3 LLM Failure

Error: API timeout or rate limit (if using external LLM)

Trigger: OpenAI/HuggingFace API unavailable

Handling:

- Catch exception in generator_node()
- Fallback to direct chunk concatenation
- Log error for monitoring
- Set confidence = "low" (partial answer)
- Notify user: "I'm experiencing technical difficulties. Here's what I found..."

8.4 Invalid User Input

Error: Empty or null query

Trigger: User presses Enter without typing

Handling:

- Validate input before processing
- Display: "Please enter a question."
- Do not call ask_question()

Error: Extremely long query (> 500 chars)

Trigger: User pastes very long text

Handling:

- Truncate to 500 characters
- Display warning: "Query truncated to 500 characters."
- Process truncated version

8.5 State Management Errors

Error: State object missing required field

Trigger: Node tries to access non-existent key

Handling:

- Use .get() method with default values
- Example: state.get("chunks", [])
- Log warning for debugging
- Continue with default value

Error: Graph execution deadlock

Trigger: Conditional routing returns invalid destination

Handling:

- Ensure should_escalate() only returns "hitl" or "end"
- Add error logging if unexpected value returned
- Default to END to prevent infinite loops

8.6 ChromaDB Errors

Error: Permission denied when writing to ./chroma_db/

Trigger: Insufficient file system permissions

Handling:

- Catch PermissionError
- Display: "Error: Cannot write to chroma_db folder. Check permissions."
- Suggest running with appropriate privileges

Error: Corrupted database

Trigger: ChromaDB fails to load existing collection

Handling:

- Catch exception during Chroma() initialization
- Display: "Error: Database corrupted. Reingest your documents."
- Provide command: "python ingest.py"

Prepared by: Rutuja Fulambarkar

Project: RAG-Based Customer Support Assistant

Organization: Innomatics Research Labs

END OF LOW-LEVEL DESIGN DOCUMENT